

Non-linear Data Structures Contd....

The Heap Data Structure $\Rightarrow$ {Give least or largest (very fast)}

{ Sometimes, we need to fetch the largest or smallest value in a collection in very quick time (Constant Time) (Big $O(1)$) In these kind of cases we generally prefer using the heap data structure. }

Normally $\Rightarrow$

2	4	8	9	11	19
---	---	---	---	----	----

 $\rightarrow$ We can retrieve a key using Binary Search at $O(\log n)$ time.

Also $\Rightarrow$

8	2	6	4	1
---	---	---	---	---

 $\rightarrow$ sort $\rightarrow$

1	2	4	6	8
---	---	---	---	---

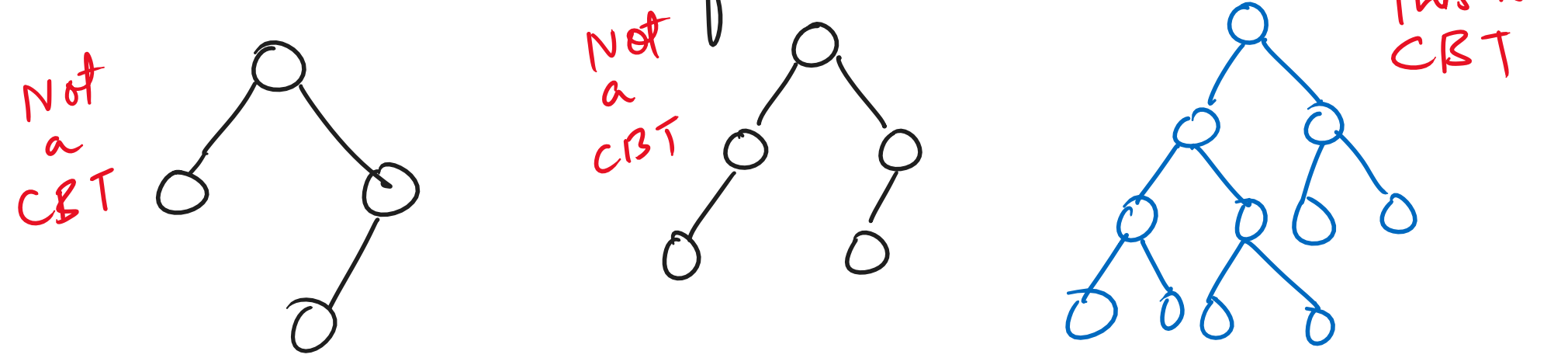
 $\rightarrow O(n \log n)$ merge sort
 $\therefore$ then we can access largest or smallest.

* A heap is called a CBT (Complete Binary Tree)

* $\rightarrow$ A CBT is a tree where insertions always happen from Left to Right in each level.

* $\rightarrow$ A level cannot be left incomplete.

* $\rightarrow$ There can be leaf nodes available.



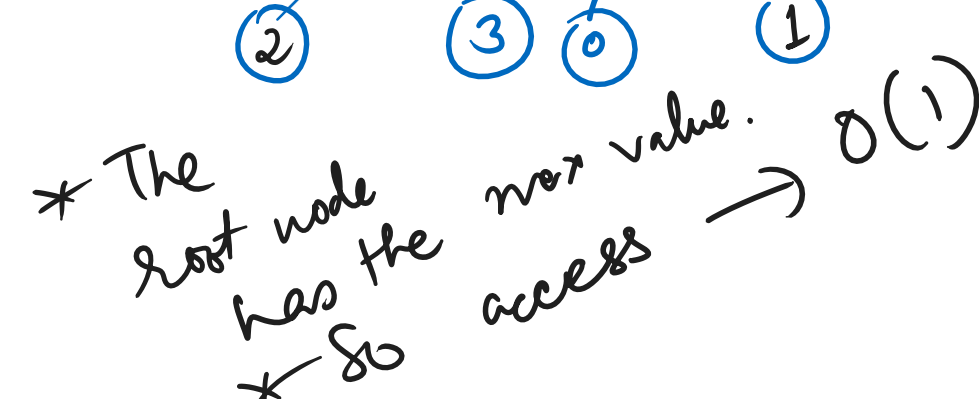
* There are two types of Heaps: Max Heap, Min Heap } (CBT)

Sample array $\rightarrow$

6	2	1	4	3	0	9
---	---	---	---	---	---	---

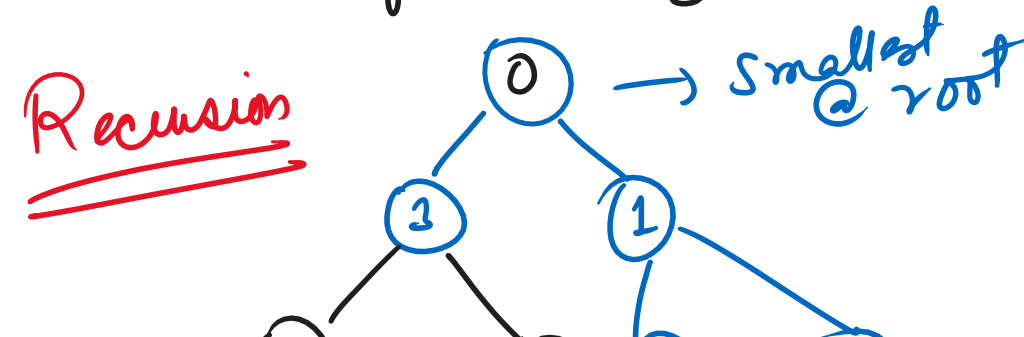
Max Heap: { $L < N > R$ }
 { Each node will be greater than its l & r children }

* Repetitive Swap if needed.



* The root node has the max value. $\rightarrow O(1)$
 * So access

Min Heap: { $L > N < R$ }
 { Each node will be less than its left & right children }



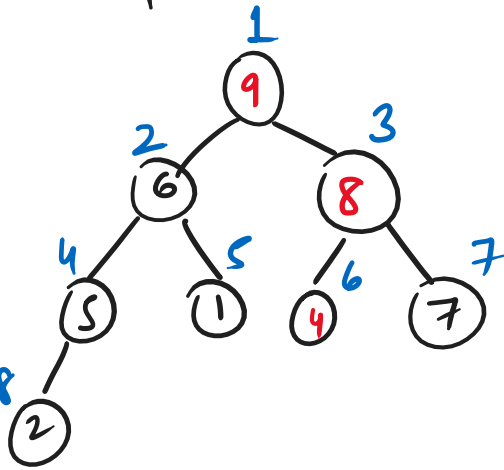
Recursion

* Convert the given array:

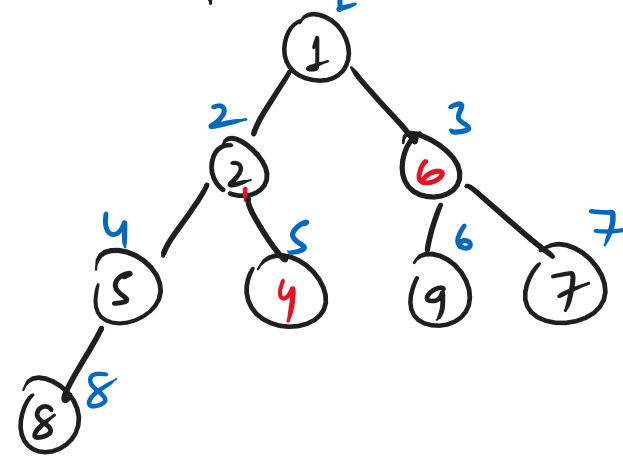
8	6	4	2	1	9	7	5
---	---	---	---	---	---	---	---

 into a Max Heap & Min Heap

Max Heap ($L < N > R$)



Min Heap ($L > N < R$)



Important Formulae

left child index (lci)

$2i$

right child index (rci)

$2i + 1$

parent index (pi)

$i/2$

Max Heap Array

-1	9	6	8	5	1	4	7	2
0	1	2	3	4	5	6	7	8

$\hookrightarrow$ Sentinel Value

Min Heap Array

-1	1	2	6	5	4	9	7	8
0	1	2	3	4	5	6	7	8

Priority Queue

* A Priority Queue is a min-heap by default.

* To convert a min-heap into a max-heap we use the following syntax $\rightarrow$

Priority Queue <Integer> maxheap = new

Priority Queue<> (Collections.reverseOrder());

* (Min/Max) $\rightarrow$ (99% Heaps) 1% (Anything else will be mentioned)

* Find the kth largest element in a given array $\rightarrow$

{ 3, 2, 1, 5, 6, 4 } { k = 3 }

0 1 2 3 4 5

1st largest = 6

2nd largest = 5

3rd largest = 4

Trick: Take a min-heap:

kth Smallest:

Max Heap

~~3~~
~~2~~
~~1~~
 3 ✓
 2 ✓
 1 ✓
 3rd Smallest

element
~~3~~
~~2~~
~~1~~
 3
 2
 1
 3rd largest $\leftarrow$ 4

size
 $6 > 3$ 5
 $5 > 3$ 4
 $4 > 3$ 3
 $3 = 3$

(It saves a lot of time)

* { 8, 6, 4, 9, 5, 2, 1 } 3rd largest $\rightarrow$

$n \log n \rightarrow$ Sort $\rightarrow$ { 1, 2, 4, 5, 6, 8, 9 } $n=7$

(0-6)
 0, 1, 2, 3, 4, 5, 6

arr[4] = 3rd largest.

[$n \log n + O(n)$] Very Slow:

Heaps $\rightarrow$ s-l or l-s

kth Smallest (Idea)

Max Heap

* Insert all element

* Remove elements till size $> k$

* The top element will be kth smallest

kth largest

Min Heap

* Insert all elements

* Remove elements till size $> k$

* The top element will be kth largest

* M $\rightarrow$ DSA
 T $\rightarrow$ DSA
 W $\rightarrow$ DSA
 T $\rightarrow$ Aptitude
 F $\rightarrow$ Aptitude
 S $\rightarrow$ Development + Core Subjects
 S $\rightarrow$ Rest

1 hr (M-23 2027)
 "
 "
 "
 "
 "
 "